

Java Advanced

Placement Training Program

Design Patterns

Design patterns are well-established, reusable solutions to recurring software design problems — a shared vocabulary for structuring code. A few patterns come up constantly in real Java code:

- Singleton — ensures a class has only one instance, globally accessible (used carefully, since overuse creates hidden global state).
- Factory — centralizes object creation logic, so callers ask for 'an object that does X' without knowing the exact concrete class being instantiated.
- Observer — lets objects subscribe to and get notified of events/changes in another object, the foundation of most event-handling systems.
- Strategy — encapsulates interchangeable algorithms behind a common interface, letting behavior be swapped at runtime without changing the code that uses it.

Streams & Lambdas

Lambda expressions (introduced in Java 8) let you write small, anonymous functions inline — `(a, b) -> a + b` — instead of verbose anonymous inner classes. The Stream API builds on lambdas to process collections in a functional, declarative style: `list.stream().filter(x -> x > 10).map(x -> x * 2).collect(Collectors.toList())` reads as a pipeline of transformations rather than a manual loop with mutable accumulator variables. This style tends to produce more concise, less error-prone code for common data-processing tasks — though understanding the equivalent imperative loop underneath still matters for debugging performance issues.

Advanced Concurrency

Beyond basic synchronized blocks, the `java.util.concurrent` package provides higher-level, safer concurrency tools: `ExecutorService` manages a pool of threads for you rather than manually creating and managing `Thread` objects; `ConcurrentHashMap` provides a thread-safe map without needing to manually synchronize every access; `CompletableFuture` represents a value that will be available asynchronously, letting you chain dependent asynchronous operations cleanly. Reaching for these higher-level tools instead of hand-rolled synchronization is almost always the right call in real production code — hand-written concurrent code is notoriously easy to get subtly wrong.

Building a Capstone Application

The capstone project for this module brings together OOP design, collections, exception handling, and (where relevant) concurrency and streams into one cohesive application — the kind of project that demonstrates not just knowing individual Java features, but knowing how to combine them into a coherent, well-structured program. Emphasis is placed on clean class design (each class with one clear responsibility), proper exception handling at the boundaries where things can actually fail (file I/O, user input, database calls), and code that a reviewer unfamiliar with the project could read and understand without extensive explanation.

Learn More

Refactoring Guru — Design Patterns

<https://refactoring.guru/design-patterns>

Baeldung — Java Guides

<https://www.baeldung.com/>

Oracle — The Java Tutorials: Concurrency

<https://docs.oracle.com/javase/tutorial/essential/concurrency/>